FIELD GUIDE • v1

SynapseGarrison

One-page guide to protecting Python agent steps and memory

1 Configure a persistent key

For cross-process signature verification, inject the same strong secret into every replica. Keep it in your platform secret manager.

```
export SYNAPSE_GARRISON_SECRET_KEY='…'
```

Use at least 16 bytes; never commit the value.

2 Know the startup behavior

If cryptography is missing, import attempts a quiet pip install. If blocked, it falls back to standard-library HMAC. This can add cold-start latency.

No configured key = ephemeral signatures (restart-loss).

3 Protect sync and async steps

Pass the agent first (or as agent=). State is captured before execution and checked afterward.

```
from synapse_garrison import garrison_protect
@garrison_protect()
def step(agent): return agent.run()
```

For async tools, use the companion decorator:

```
from synapse_garrison import garrison_protect_async
@garrison_protect_async()
async def step(agent): return await agent.run()
```

4 Write memory through the sandbox

Explicit writes are signed before appending. Mark untrusted origins accurately; origin is part of the HMAC envelope.

```
from synapse_garrison import SynapseSandbox
with SynapseSandbox(agent) as guard:
block = guard.write_memory(page_text,
origin="Origin: External_Web_Scrape")
assert guard.key_manager.verify_integrity(block)
```

No registered memory list? The signed block stays sandbox-local.

5 Match fields to your agent model

Defaults cover common prompt/role, memory, and configuration names. Override the explicit arrays when your framework differs.

```
policy = SecurityPolicy(
identity_fields=("prompt", "role"),
monitored_memory_fields=("memory_store",),
)
@garrison_protect(policy=policy)
def step(agent): ...
```

6 Run the built-in demo

From the directory containing the file:

```
python synapse_garrison.py
```

The demo checks benign sync + async steps, a verified local log, adversarial prompt injection rollback, and tampered-signature rejection.

Security notes & troubleshooting

- Rollback restores Python object state only; it cannot undo network calls, tool actions, database writes, or other external side effects.
- HMAC proves integrity under the key, not that memory content is trustworthy. Keep tool permissions least-privileged and gate consequential actions.
- If rollback encounters an uncopyable live client/socket, its snapshot is sanitized; the live object may need to be recreated after rollback.
- For rejected writes, inspect the origin and policy fields. For cross-node verification failures, ensure identical key bytes and aliases on every instance.